

EECS 482

Lab 3: RAI, Exceptions

Administrivia

Today: 05/30

Project 1 Due 6/2

Project 2 Released Next
Week (after P1 due) &
Due 06/20 (in ~3 weeks)

++ Difficulty vs. P1
Start Early!!

Agenda

1. Q1: Luxury Box
2. RAII
3. Exceptions
4. Q2: `lock_guard` Exercise

— — —

Q1: Luxury Box

Luxury Box: Q2



BIG



Luxury Box: Q2

Only fans from the same conference can be in the box at once.

Implement using monitors.

```
int active_big10 = 0, active_pac12 = 0; // fans in the box

void big10_wants_to_enter();
void big10_leaves();

void pac12_wants_to_enter();
void pac12_leaves();
```

Let's code it! (write just the
pac12 functions)

Luxury Box: Q2

```
cv pac12;  
cv big10;  
mutex box_lock;
```

What if *just one* Big10 fan is in the box, but Pac12 fans keep showing up?

```
void pac12_wants_to_enter() {  
    box_lock.lock();  
    while (active_big10 > 0) {  
        pac12.wait(box_lock);  
    }  
    active_pac12++;  
    box_lock.unlock();  
}
```

```
void pac12_leaves() {  
    box_lock.lock();  
    active_pac12--;  
    if (active_pac12 == 0) {  
        big10.broadcast();  
    }  
    box_lock.unlock();  
}
```

Starvation

When threads are not able to accomplish their work because other threads have control.

How can we fix this in the context of the luxury box problem?

TURN controls who can enter the box

**What if there are
no Big10 fans, and
it's their turn?**

```
enum Conference {BIG10, PAC12};  
Conference TURN = PAC12;
```

```
void pac12_wants_to_enter() {  
    box_lock.lock();  
    while (active_big10 ||  
           TURN == BIG10) {  
        pac12.wait(box_lock);  
    }  
    active_pac12++;  
    box_lock.unlock();  
}
```

```
void pac12_leaves() {  
    box_lock.lock();  
    active_pac12--;  
    if (active_pac12 == 0) {  
        big10.broadcast();  
    }  
    box_lock.unlock();  
}
```

TURN controls who can enter the box

When to switch the
TURN variable?

`enum Conference {BIG10, PAC12};`
`Conference TURN = PAC12;`

```
void pac12_wants_to_enter() {  
    box_lock.lock();  
    waiting_pac12++;  
    while (active_big10 ||  
           (TURN == BIG10 &&  
            waiting_big10)) {  
        pac12.wait(box_lock);  
    }  
    active_pac12++;  
    waiting_pac12--;  
    box_lock.unlock();  
}
```

```
void pac12_leaves() {  
    box_lock.lock();  
    active_pac12--;  
    if (active_pac12 == 0) {  
        big10.broadcast();  
    }  
    box_lock.unlock();  
}
```

Switch turn when last fan leaves?

Still have starvation;
PAC12 fans could “hand
off” box since a single
PAC12 fan being in the
box makes all BIG10 wait

```
enum Conference {BIG10, PAC12};  
Conference TURN = PAC12;
```

```
void pac12_wants_to_enter() {  
    box_lock.lock();  
    waiting_pac12++;  
    while (active_big10 ||  
           (TURN == BIG10 &&  
            waiting_big10)) {  
        pac12.wait(box_lock);  
    }  
    active_pac12++;  
    waiting_pac12--;  
    box_lock.unlock();  
}
```

```
void pac12_leaves() {  
    box_lock.lock();  
    active_pac12--;  
    if (active_pac12 == 0) {  
        big10.broadcast();  
        TURN = BIG10;  
    }  
    box_lock.unlock();  
}
```

Switch turn when fan enters?

**Fair, but inefficient;
there can only be one fan
in the box at a time**

```
enum Conference {BIG10, PAC12};  
Conference TURN = PAC12;
```

```
void pac12_wants_to_enter() {  
    box_lock.lock();  
    waiting_pac12++;  
    while (active_big10 ||  
           (TURN == BIG10 &&  
            waiting_big10)) {  
        pac12.wait(box_lock);  
    }  
    active_pac12++;  
    waiting_pac12--;  
    TURN = BIG10;  
    box_lock.unlock();  
}
```

```
void pac12_leaves() {  
    box_lock.lock();  
    active_pac12--;  
    if (active_pac12 == 0) {  
        big10.broadcast();  
    }  
    box_lock.unlock();  
}
```

Switch turn when any fan leaves?

**Better, but individual fans
might still be starved**

```
enum Conference {BIG10, PAC12};  
Conference TURN = PAC12;
```

```
void pac12_wants_to_enter() {  
    box_lock.lock();  
    waiting_pac12++;  
    while (active_big10 ||  
           (TURN == BIG10 &&  
            waiting_big10)) {  
        pac12.wait(box_lock);  
    }  
    active_pac12++;  
    waiting_pac12--;  
    box_lock.unlock();  
}
```

```
void pac12_leaves() {  
    box_lock.lock();  
    active_pac12--;  
    TURN = BIG10;  
    if (active_pac12 == 0) {  
        big10.broadcast();  
    }  
    box_lock.unlock();  
}
```

Key Takeaways from Luxury Box

— — —

- There are many factors involved when coding with monitors
- Efficiency is important
 - Ask yourself "how many CVs should I use?"
- Fairness should also be considered
 - Difficult to analyze and not as objective
- Evaluating these tradeoffs is key
 - **There is often not one perfect solution**
- **The most important factor is always correctness**
 - Start with a correct solution, and gradually improve it

Resource Acquisition Is Initialization (RAII)

What is a Resource?

— — —

- Can be “acquired” and “released”
- Usually expensive to hold
- Examples:
 - Memory (new, delete)
 - Mutex (lock, unlock)
 - File (open, close)

Variable Lifetime

- Tied to the scope it is defined in
- Constructor is called where the variable is introduced into the scope
- When a scope exits the destructors of all contained variables are called in the *reverse* construction order

```
struct A {  
    A() { // Constructor  
        std::cout << "A ctor" << std::endl;  
    }  
  
    ~A() { // Destructor  
        std::cout << "A dtor" << std::endl;  
    }  
};  
  
int main() { A a; }
```

This will print:

A ctor

A dtor

Note: the lifetime of “a”
is bound to the function
“main”

RAII

Idea: Use variable lifetimes to manage resources

- Make a class to hold the resource
- Acquire/release the resource with the constructor/destructor

Benefits:

- Resources automatically released when exiting scope
 - function return
 - thrown exception
- No worries about manually releasing

Example where RAII would be useful

```
int do_thing() { // Return 0 on success, -1 on failure
    m.lock();
    if (shared.action_that_might_fail_and_return_neg1() < 0) {
        m.unlock(); // <-----
        return -1;
    }
    if (shared.another_action_that_might_fail() < 0) {
        m.unlock(); // <-----
        return -1;
    }
    m.unlock(); // <-----
    return 0;
}
```

A) What if you forget to unlock m??
(correctness is jeopardized)

B) This is ugly and bad style

Solution: Use an RAI class

```
int do_thing() { // Return 0 on success, -1 on failure
    // lock_guard calls lock() in ctor, unlock() in dtor
    lock_guard lock{m};
    if (shared.action_that_might_fail_and_return_neg1() < 0) {
        return -1; // lock exits scope, unlocks m
    }
    if (shared.another_action_that_might_fail() < 0) {
        return -1; // lock exits scope, unlocks m
    }
    if (...) { return -1; }
    return 0; // lock exits scope, unlocks m
}
```

Solution: Use an RAI class

```
class mutex_guard {  
public:  
    // acquire resources in constructor  
    mutex_guard(mutex &in) : my_mutex{in} {  
        my_mutex.lock();  
    }  
    // release resources in destructor  
    ~mutex_guard() {  
        my_mutex.unlock();  
    }  
private:  
    mutex &my_mutex;  
};
```

Exceptions

Example

```
#include <stdexcept>
#include <iostream>

int main() {
    try {
        int x;
        std::cin >> x;
        if (x == 0) {
            throw std::runtime_error("received 0");
        }
    } catch (std::runtime_error &e) {
        std::cout << e.what() << std::endl; // if runs, prints "received 0"
    }
}
```

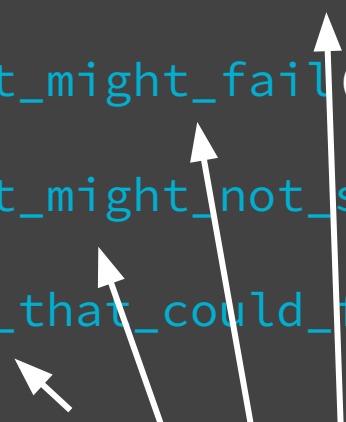
Error handling using return value

```
int do_thing() { // Return 0 on success, -1 on failure
    if (action_that_might_fail_and_return_neg1() != 0) {
        return -1;
    }
    if (another_action_that_might_fail() != 0) {
        return -1;
    }
    if (another_action_that_might_not_succeed() != 0) {
        return -1;
    }
    return 0;
}
```

What if we added **another** action that might fail?

Error handling using return value

```
int do_thing() { // Return 0 on success, -1 on failure
    if (action_that_might_fail_and_return_neg1() != 0) {
        return -1;
    } if (another_action_that_might_fail() != 0) {
        return -1;
    } if (another_action_that_might_not_succeed() != 0) {
        return -1;
    } if (yet_another_action_that_could_fail() != 0) {
        return -1;
    }
    return 0;
}
```



What if these **threw exceptions** on failure, instead of returning -1?

Error handling using exceptions

```
void do_thing() {  
    try {  
        action_that_might_fail_and_throw_exception();  
        another_action_that_might_fail();  
        another_action_that_might_not_succeed();  
    } catch (exception_type_1 &e) {  
        error_handler_1(e);  
    } catch (exception_type_2 &e) {  
        error_handler_2(e);  
    }  
}
```

What if we added **another** action that might fail?

Error handling using exceptions

```
void do_thing() {  
    try {  
        action_that_might_fail_and_throw_exception();  
        another_action_that_might_fail();  
        another_action_that_might_not_succeed();  
        yet_another_action_that_might_not_succeed();  
    } catch (exception_type_1 &e) {  
        error_handler_1(e);  
    } catch (exception_type_2 &e) {  
        error_handler_2(e);  
    }  
}
```

Exceptions without RAll (bad)

```
mutex m;
```

```
void do_thing() try {
```

```
    m.lock();
```

```
    action_that_might_fail_and_return_neg1();
```

```
    another_action_that_might_fail();
```

```
    another_action_that_might_not_succeed();
```

```
    yet_another_action_that_could_fail();
```

```
    m.unlock();
```

```
} catch (exception_type &e) {
```

```
    m.unlock();
```

```
    error_handler(e);
```

```
}
```

pairing 1 lock with 2
unlocks :(



Exceptions with RAII (good)

```
mutex m;
```

```
void do_thing() try {  
    mutex_guard guard{m};  
    action_that_might_fail_and_return_neg1();  
    another_action_that_might_fail();  
    another_action_that_might_not_succeed();  
    yet_another_action_that_could_fail();  
} catch (exception_type &e) {  
    error_handler(e);  
}
```

if exception is
thrown, guard is
destroyed :)

Q2: RAII Exercise

Q2: RAII Exercise

1. Implement RAII interface for locks
2. Convert example monitor program

RAII Interface: Better lock guard

— — —

```
class lock_guard {  
    mutex* m = nullptr;  
public:  
    lock_guard(mutex& m_in)  
        : m(&m_in) {  
        m->lock();  
    }  
  
    ~lock_guard() {  
        if (m) m->unlock();  
    }  
  
}
```

```
// to do things properly  
// disallow copies, and implement move  
lock_guard(lock_guard const&) = delete;  
lock_guard& operator=(lock_guard const&) = delete;  
  
lock_guard(lock_guard&& other) noexcept  
    : m { other.m } {  
    // leaves other guard in a safe state  
    other.m = nullptr;  
}  
  
lock_guard& operator=(lock_guard&& other) noexcept {  
    if (this != &other) {  
        m = other.m;  
        other.m = nullptr;  
    }  
    return *this;  
}
```

RAII Interface: Changed Program

— — —

```
void child(uintptr_t arg) {
    auto message =
reinterpret_cast<char*>(arg);

    lock_guard guard(mutex1);

    cout << "child called with
message " << message << ", setting
child_done = 1"
        << endl;
    child_done = 1;
    cv1.signal();
}
```

```
void parent(uintptr_t arg) {
    {
        lock_guard guard(mutex1);
        cout << "parent called with arg " << arg << endl;
    }

    thread t1(child, reinterpret_cast<uintptr_t>("test message"));
    {
        lock_guard guard(mutex1);
        while (!child_done) {
            cout << "parent waiting for child to run\n";
            cv1.wait(mutex1);
        }
        cout << "parent finishing" << endl;
    }
}
```

Questions?